

Chapter 2: An Overview of Collections

Fundamentals of Python: Data Structures, Second Edition

Learning Objectives

- Define the four general categories of collections – linear, hierarchical, graph, and unordered
- List the specific types of collections that belong to each of the four categories of collections
- Recognize which collections are appropriate for given applications
- Describe the commonly used operations on every type of collection
- Describe the difference between an abstract collection type and its implementations

An Overview of Collections

- Collection
 - A group of zero or more items that can be treated as a conceptual unit
- All collections have the same fundamental purposes:
 - They help programmers effectively organize data in programs
 - They also help model the structure and behavior of objects in the real world
- Collections can be viewed from two perspectives:
 - Users or clients of collections are concerned with what they do in various applications
 - Developers or implementers are concerned with how they can best perform as general-purpose resources

Collection Types (1 of 2)

- Built-in collection types include
 - String, list, tuple, set, and dictionary
 - String and list are the most common and fundamental types
- Other important types of collections include
 - Stacks, queues, priority queues, binary search trees, heaps, graphs, bags, and various types of sorted collections
- Collections can be
 - Homogeneous – all items in the collection must be of the same type
 - Heterogeneous – items can be of different types

Collection Types (2 of 2)

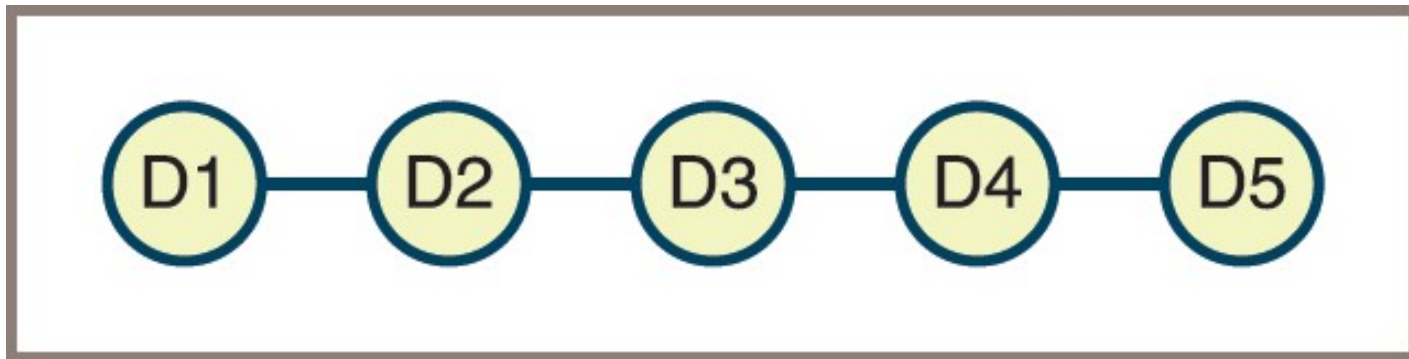
- Collections are typically dynamic rather than static:
 - Can grow or shrink with the needs of the problem
 - Contents can change throughout the course of a program
- Exception to the above
 - Immutable collection – such as Python's string or tuple
 - Items are added during its creation and no items can be added, removed, or replaced later
- Categories of collections include
 - Linear, hierarchical, graph, unordered, and sorted

Linear Collections (1 of 2)

- Items in a linear collection are ordered by position:
 - Each item (except the first) has a unique predecessor
 - Each item (except the last) has a unique successor

Linear Collections (2 of 2)

Figure 2-1: A linear collection

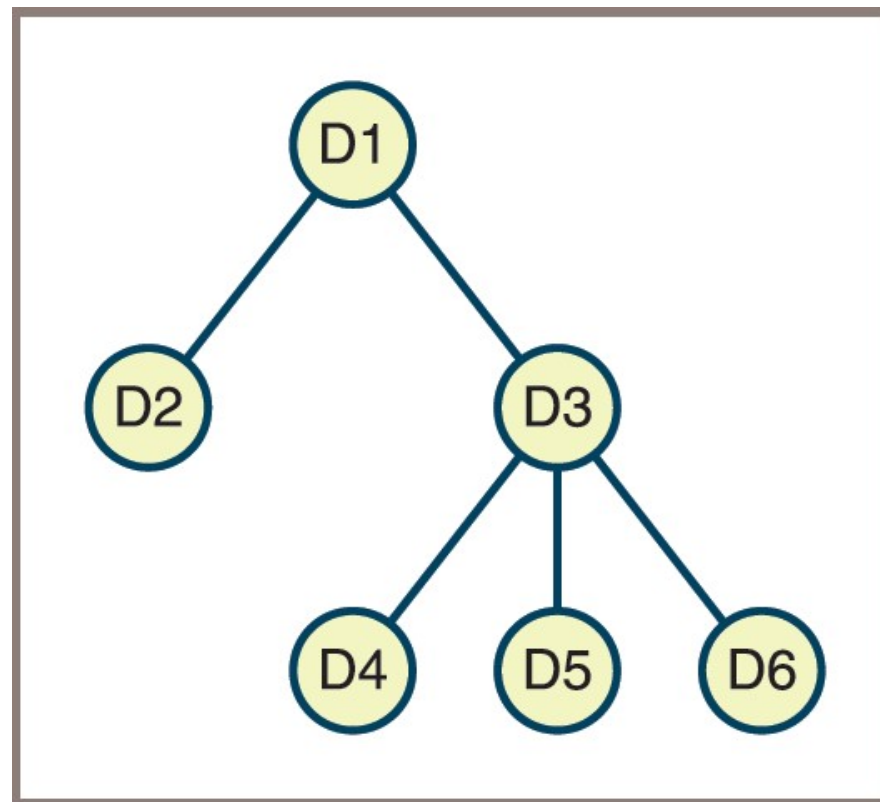


Hierarchical Collections (1 of 2)

- Data items are ordered in a structure resembling an upside-down tree:
 - Each data item (except the top) has just one predecessor, called its parent, but potentially many successors, called its children

Hierarchical Collections (2 of 2)

Figure 2-2: A hierarchical collection

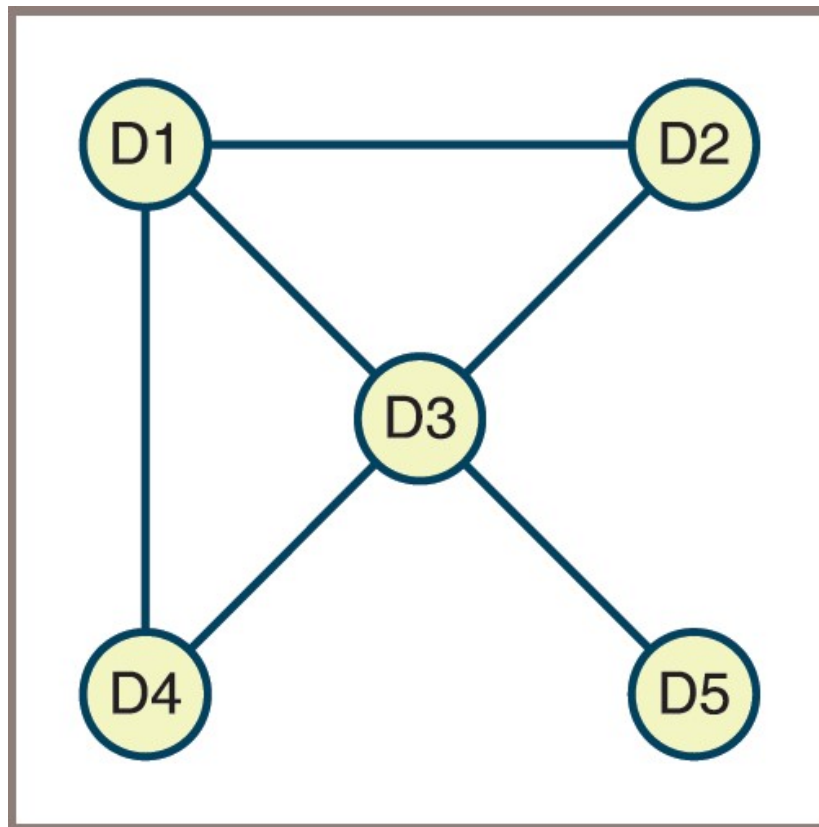


Graph Collections (1 of 2)

- Each data item can have many predecessors and many successors

Graph Collections (2 of 2)

Figure 2-3: A graph collection

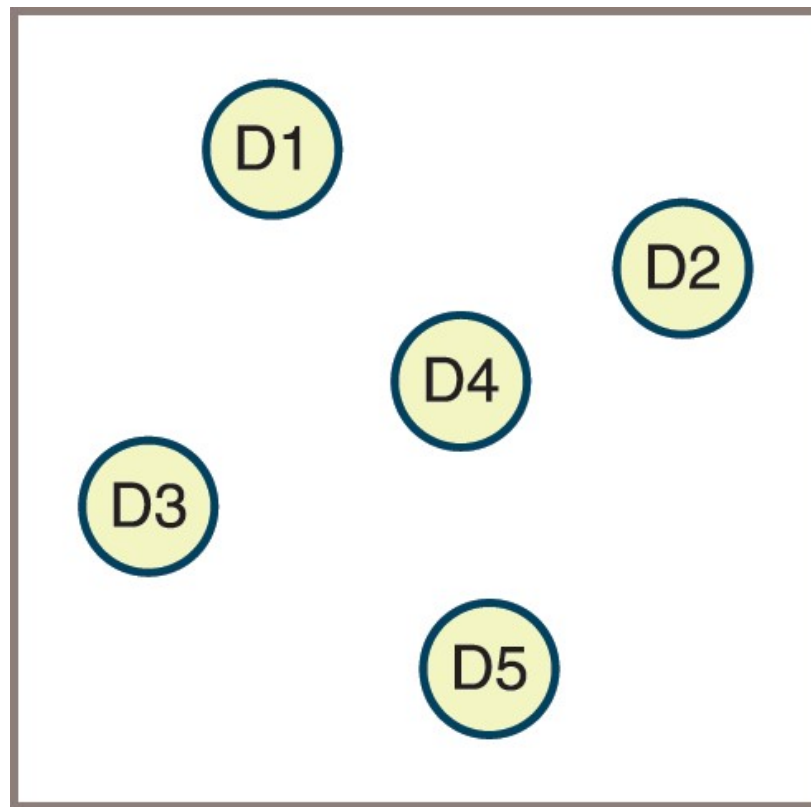


Unordered Collections (1 of 2)

- Items are not in any particular order:
 - Not possible to meaningfully speak of an item's predecessor or successor
- Example
 - A bag of marbles

Unordered Collections (2 of 2)

Figure 2-4: An unordered collection

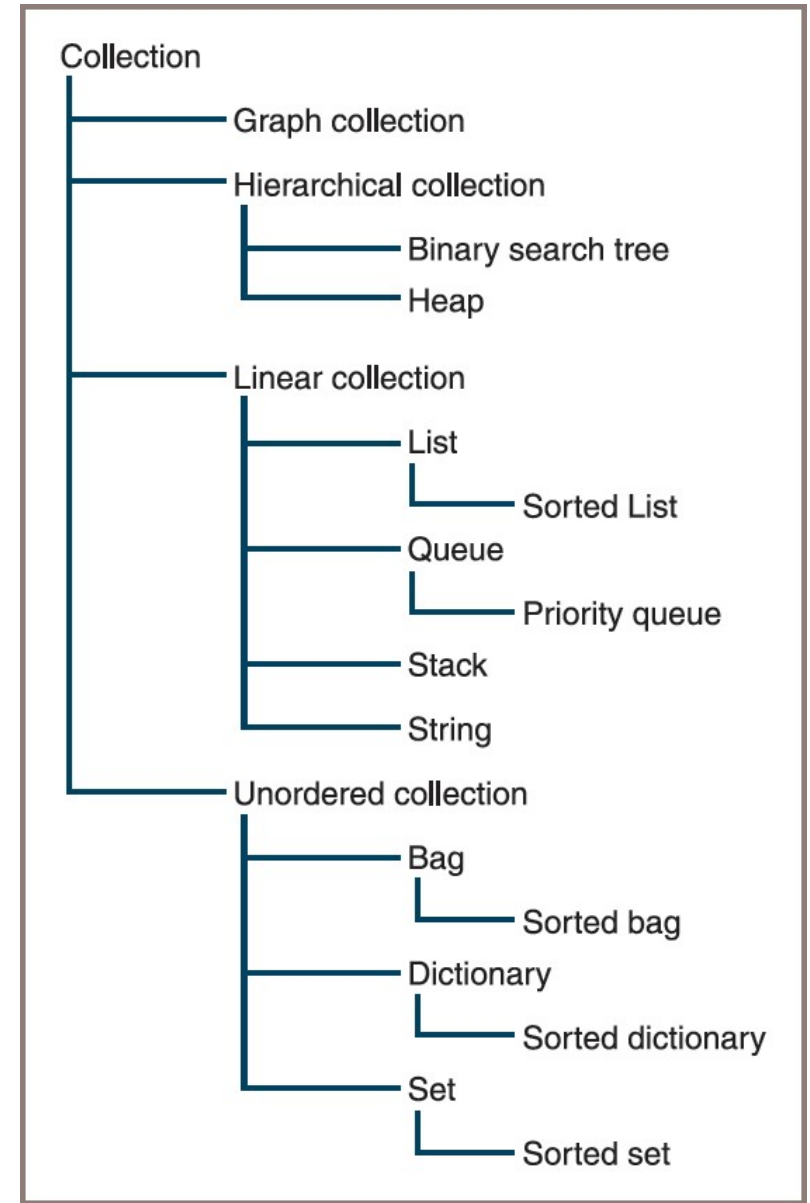


Sorted Collections

- Imposes a natural ordering on its items
- Examples
 - Entries in a phone book and the names on a class roster
- Must be some rule for comparing items
- Sorted list is the most common example of a sorted collection
- A special type of hierarchical collection also imposes a natural ordering on its items,
 - Known as a **binary search tree**
- Sorted collections allow clients to visit all of its items in sorted order

A Taxonomy of Collection Types

Figure 2-5: A taxonomy of collection types



Operations on Collections (1 of 2)

Category of Operation	Description
Determine the size	Use Python's len function to obtain the number of items currently in the collection.
Test for item membership	Use Python's in operator to search for a given target item in the collection. Returns True if the item is found, or False otherwise.
Traverse the collection	Use Python's for loop to visit each item in the collection. The order in which the items are visited depends upon the type of collection.
Obtain a string representation	Use Python's str function to obtain the string representation of the collection.
Test for equality	Use Python's == operator to determine whether two collections are equal. Two collections are equal if they are of the same type and contain the same items. The order in which pairs of items are compared depends on the type of collection.

Operations on Collections (2 of 2)

Category of Operation	Description
Concatenate two collections	Use Python's + operator to obtain a new collection of the same type as the operands, and containing the items in the two operands.
Convert to another type of collection	Create a new collection with the same items as a source collection. Cloning is a special case of type conversion, where the two collections are of the same type.
Insert an item	Add the item to the collection, possibly at a given position.
Remove an item	Remove the item from the collection, possibly at a given position.
Replace an item	Combine removal and insertion into one operation.
Access or retrieve an item	Obtain an item, possibly at a given position.

Fundamental Operations on All Collection Types

- Note that several of these operations are associated with standard Python operators, functions, or control statements:
 - Such as `in`, `+`, `len`, `str`, and the `for` loop
- There is no single name for
 - Insertion, removal, replacement, or access operations in Python
- There are a few standard variations:
 - The method `pop` is used to remove items at given positions from a Python list or values at given keys from a Python dictionary
 - The method `remove` is used to remove given items from a Python set or a Python list

Type Conversion (1 of 2)

- You might already know about type conversion from its use in the input of numbers:
 - Convert a string of digits from the keyboard to an **int** or **float** by applying the **int** or **float** function to the input string
- You can convert one type of collection to another type of collection in a similar manner:
 - Converting a Python string to a Python list and a Python list to a Python tuple:

```
>>> message = "Hi there!"
>>> lyst = list(message)
>>> lyst
['H', 'i', ' ', 't', 'h', 'e', 'r', 'e', '!']
>>> toouple = tuple(lyst)
>>> toouple
('H', 'i', ' ', 't', 'h', 'e', 'r', 'e', '!')
```

Type Conversion (2 of 2)

- Creating a list from a range:

```
>>> lyst = list(range(1, 11, 2))  
>>> lyst  
[1, 3, 5, 7, 9]
```

- Other functions expect more specific types of iterable objects as arguments,
 - Such as a list of (key, value) tuples
- If an argument is omitted,
 - A collection's type conversion function returns a new, empty collection of that type

Cloning and Equality (1 of 2)

- Cloning
 - A special case of type conversion
 - Returns an exact copy of the argument to the conversion function
- Example – this code segment makes a copy of a list and then compares the two lists using the **is** and **==** operators:

```
>>> lyst1 = [2, 4, 8]
>>> lyst2 = list(lyst1)
>>> lyst1 is lyst2
False
>>> lyst1 == lyst2
True
```

Cloning and Equality (2 of 2)

- The `list` function makes a *shallow copy* of its argument list
- The items in the previous code are not themselves cloned before being added to the new list:
 - Instead, references to these objects are copied
- When collections share mutable items,
 - Side effects can occur
 - To prevent this from happening, the programmer can create a *deep copy* by writing a for loop over the source collection
 - This explicitly clones its items before adding them to the new collection

Iterators and Higher-Order Functions

- Each type of collection supports an iterator or **for** loop:
 - The order in which the **for** loop serves up a collection's items depends on the manner in which the collection is organized
- The **for** loop plays a useful role in the implementation of several other basic collection operations, such as
 - **+**, **str**, and **type** conversions
- It also plays a role in standard Python functions, such as
 - **Sum**, **max**, and **min**
- And also supports the use of higher-order functions, such as
 - **Map**, **filter**, and **reduce**
 - These functions can be used with any type of collection, not just lists

Implementations of Collections (1 of 3)

- Programmers who use collections need to know how to instantiate and use each type of collection
- From a user's perspective, a collection is an abstraction:
 - For this reason, collections are also called abstract data types (ADTs)
- Developers of collections
 - Are concerned with implementing a collection's behavior in the most efficient manner possible
- Some programming languages provide only one implementation of each of the available collection types:
 - Python is one of those
 - Java provides several

Implementations of Collections (2 of 3)

- One aim of this book is to give Python programmers the same options as the Java programmer
 - As well as introduce abstract collection types and their implementations that are not available in either language
- For each category of collection,
 - You'll see one or more abstract collection types and one or more implementations of each type
- A software system is often built layer by layer
 - With each layer treated as an abstraction by the layers above that utilize it
- Without abstraction,
 - You would need to consider all aspects of a software system simultaneously

Implementations of Collections (3 of 3)

- In Python,
 - Functions and methods are the smallest units of abstraction
 - Classes are next in size
 - Modules are the largest
- This text implements abstract collection types as classes or sets of related classes in modules
- General techniques for organizing these classes are covered in Chapter 5 and Chapter 6

Chapter Summary (1 of 2)

- Collections are objects that hold zero or more objects:
 - A collection has operations for accessing its objects, inserting them, removing them, determining the collection's size, and traversing or visiting the collection's objects
- The five main categories of collections are linear, hierarchical, graph, unordered, and sorted
- Linear collections order their items by position, with each but the first having a unique predecessor and each but the last having a unique successor
- With one exception, the items in a hierarchical collection have a unique predecessor and zero or more successors:
 - A single item called the root has no predecessor

Chapter Summary (2 of 2)

- The items in a graph can have zero or more successors and zero or more predecessors
- The items in an unordered collection are in no particular order
- Collections are iterable—each item contained within a collection can be visited with a **for** loop:
 - The programmer can also use the higher-order functions **map**, **filter**, and **reduce** to simplify data processing with collections
- An abstract data type is a set of objects and operations on those objects:
 - Collections are thus abstract data types
- A data structure is an object that represents the data contained in a collection